Software Engineering Lab

Lab 30

Leader Election dengan Fencing Tokens

Mencegah Split-Brain & Stale Writes
Menggunakan Monotonic Revision

Lukman / lukman-ss

Masalah Mendasar: Transient Split-Brain

Ketika pause eksekusi melampaui batas waktu lease koordinator

BATAS KENDALI TUNGGAL

Sistem terdistribusi menuntut tepat satu primary worker untuk job eksekusi kritis
Lease-based leader election membatasi masa jabatan pemimpin via Time-To-Live (TTL)
Leader wajib memperbarui sewa secara periodik sebelum TTL habis (heartbeat renewal)

ANOMALI ASYNCHRONOUS PAUSE

Stop-the-world GC pause atau partisi jaringan dapat menghentikan node lebih lama dari TTL
Koordinator menganggap leader mati dan melimpahkan kepemimpinan ke node lain
Node lama terbangun tanpa sadar bahwa masa sewanya telah hangus di koordinator

Dua node meyakini dirinya sebagai leader yang sah secara bersamaan (dual-leader window)

Tulisan lama dari node yang tertunda berisiko menimpa mutasi baru di shared storage

Lease temporal semata tidak cukup menjamin mutual exclusion mutlak tanpa proteksi token

A lease grants time-bounded authority; uncoordinated pauses cause silent split-brain writes.

Mental Model: Pintu Akses & Kartu Bernomor

Mengubah kewenangan berbasis waktu menjadi urutan angka monotonik

ANALOGI KARTU AKSES ELEKTRONIK

Bayangkan pintu kamar hotel dengan kunci kartu digital sewaan berbatas jam
Penyewa A lupa memperpanjang sewa tepat waktu karena tertidur di luar ruangan
Resepsionis (koordinator) menerbitkan kartu baru untuk Penyewa B dengan nomor urut 2

PENOLAKAN DI GAGANG PINTU

Penyewa A terbangun membawa kartu nomor 1 dan mencoba membuka pintu kamar
Sensor pintu (shared resource) mencatat angka terakhir yang masuk adalah nomor 2
Pintu menolak akses kartu 1 secara instan karena nomornya kalah urutan

Setiap akuisisi sewa baru wajib menaikkan angka token revision secara strictly increasing

Resource bersama menolak setiap request dengan incomingToken <= lastSeenToken

Keamanan ditegakkan pada gerbang data, bukan pada persepsi internal klien

A fencing token converts temporal leases into monotonic ordering enforced by the storage gate.

Tiga Pilar Pola Fencing Tokens

Pemisahan tanggung jawab antara koordinasi, kandidat, dan proteksi resource

1. LINEARIZABLE COORDINATOR

Menyimpan mapping key-lease dan counter revision monotonik global
Memberikan hak sewa berdurasi TTL dan menerbitkan fencing token baru
Menolak klaim tumpang tindih selama lease lama masih aktif (ErrLeaseHeld)

2. ELECTION CANDIDATE (NODE)

Menjalankan election loop reaktif: Acquire saat follower, Renew saat leader
Menyimpan status internal (FOLLOWER vs LEADER) dan salinan lease lokal
Meneruskan fencing token pada setiap payload penulisan ke shared resource

Menyimpan state data bersama dan melacak variabel lastSeenToken

Menjalankan validasi check-and-set atomik sebelum mutasi diizinkan

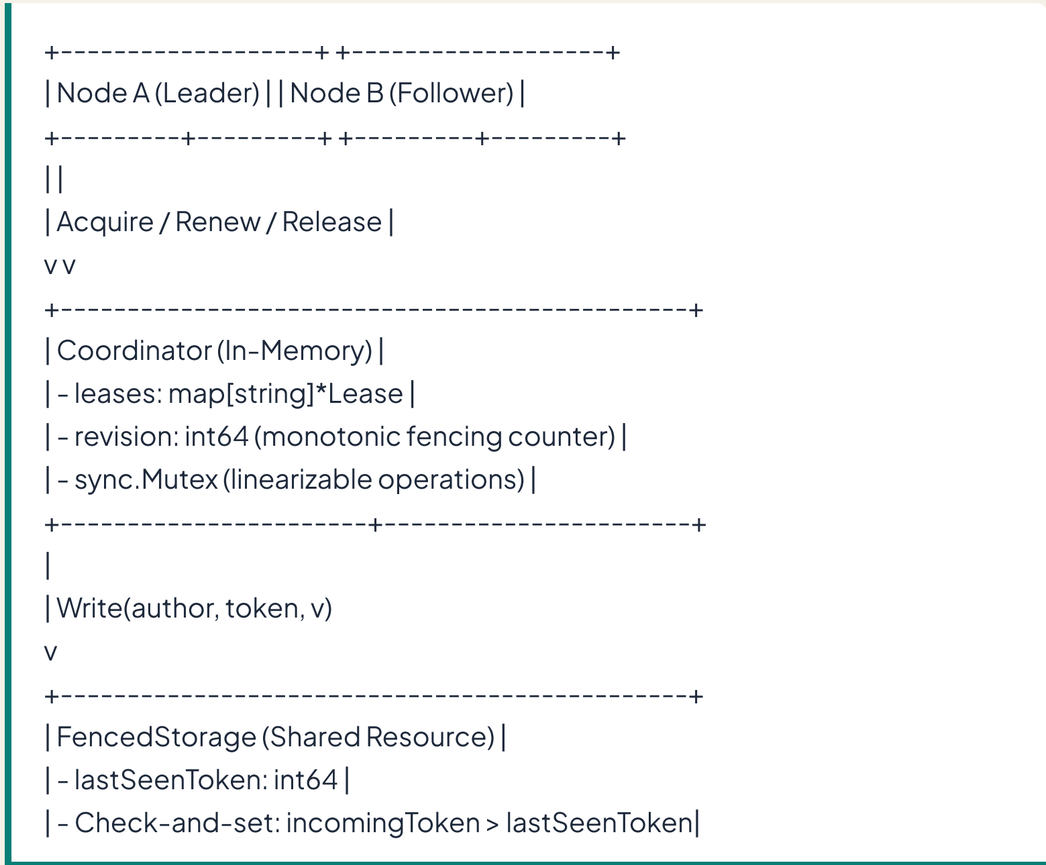
Menolak paket transaksi basi berstatus stale dengan ErrStaleFencingToken

Coordination issues tokens, candidates campaign and execute, storage enforces strict ordering.

Topologi Sistem & Alur Interaksi Komponen

Hubungan relasional antara Node A, Node B, Coordinator, dan Storage

DIAGRAM TOPOLOGI KOMPONEN



Komunikasi antara Node dan Coordinator terisolasi dari jalur penulisan data
Node tidak pernah mengkoordinasikan konsistensi data secara langsung antar rekan (peer)
Shared resource bertindak sebagai benteng pertahanan terakhir konsistensi
Seluruh komponen berjalan thread-safe dalam runtime Go murni tanpa dependensi daemon eksternal

Storage is the ultimate arbiter; nodes never negotiate consistency directly with peers.

Siklus Hidup Kandidat: Follower & Leader

Transisi state deterministik berbasis heartbeat renewal dan akuisisi

KONDISI FOLLOWER

State awal setiap node kandidat yang bergabung dalam kluster
Pada setiap tick interval renewal, memanggil coord.Acquire(key, id, ttl)
Jika sukses memperoleh lease, beralih ke state LEADER dan menyimpan token
Jika gagal (ErrLeaseHeld), tetap berstatus FOLLOWER dan menunggu tick berikutnya

KONDISI LEADER

Bertanggung jawab mengeksekusi operasi bisnis dan penulisan berfencing
Secara periodik memanggil coord.Renew(key, id, token, ttl) sebelum batas waktu habis
Jika Renew berhasil, batas waktu kadaluwarsa diperpanjang dengan token tetap
Jika Renew gagal (ErrLeaseExpired / ErrNotLeaseOwner), langsung revert ke FOLLOWER

Panggilan SimulatePause() menahan loop election tanpa mengirim heartbeat

Node tidak sempat menurunkan statusnya sendiri karena thread dalam keadaan terhenti

Heartbeat renews keep leaders active; missed renewals force an immediate step-down to follower.

Penerbitan Token Monotonik di Koordinator

Struktur Lease dan increment atomik revision di bawah sync.Mutex

```
type Lease struct {
    Key          string
    HolderID     string
    FencingToken int64
    ExpiresAt    time.Time
    TTL          time.Duration
}
```

```
c.revision++
lease := &Lease{
    Key:          key,
    HolderID:     holderID,
    FencingToken: c.revision,
    ExpiresAt:    now.Add(ttl),
    TTL:          ttl,
}
c.leases[key] = lease
```

Operasi revision++ dilindungi sync.Mutex koordinator secara ketat

Setiap lease baru dijamin memiliki angka FencingToken yang lebih tinggi

Salinan nilainya didistribusikan ke pemegang baru sebagai bukti kewenangan

Every newly granted lease atomically increments a global revision counter.

Pemeriksaan Masa Berlaku & Kepemilikan Sewa

Mekanisme Acquire mencegah klaim ganda selama lease masih aktif

```
now := time.Now().Add(c.clockOffset)
existing, found := c.leases[key]
if found && now.Before(existing.ExpiresAt) {
    if existing.HolderID == holderID {
        existing.ExpiresAt = now.Add(ttl)
        return copyLease(existing), nil
    }
    return nil, ErrLeaseHeld
}
```

Jika pemanggil adalah pemilik yang sama, expiry diperbarui tanpa menaikkan revision

Hal ini menjaga stabilitas nomor token selama pemegang kepemimpinan tidak berganti

Node kompetitor yang mencoba mengambil alih saat lease aktif akan ditolak mentah-mentah

Menjamin kepemimpinan tunggal pada kondisi normal tanpa partisi

Active leases block foreign acquisitions but permit idempotent extensions by the rightful holder.

Mekanisme Perpanjangan Sewa (Renew)

Validasi ganda: masa berlaku waktu dan pencocokan kepemilikan token

```
if now.After(existing.ExpiresAt) || now.Equal(existing.ExpiresAt) {
    delete(c.leases, key)
    return nil, ErrLeaseExpired
}
if existing.HolderID != holderID || existing.FencingToken != token {
    return nil, ErrNotLeaseOwner
}
existing.ExpiresAt = now.Add(ttl)
```

Jika now >= ExpiresAt, lease langsung dihapus dari map dan dikembalikan ErrLeaseExpired

Kandidat yang telat memperpanjang kehilangan status sewa seketika

Validasi existing.FencingToken != token menolak usaha renewal dari leader purba

Mencegah node yang terlambat bangun merebut kembali lease yang sudah dialihkan

Renewals require both an unexpired deadline and an exact match of the holder and token.

Shared Storage Check-and-Set Gate

Benteng pertahanan penolakan mutasi stale berbasis perbandingan token

```
func (s *FencedStorage) Write(author string, token int64, value string) error {
    s.mu.Lock(); defer s.mu.Unlock()
    if token <= s.lastSeenToken {
        return fmt.Errorf("%w: token %d <= last seen %d (author: %s)",
            ErrStaleFencingToken, token, s.lastSeenToken, author)
    }
    s.lastSeenToken = token
    s.records = append(s.records, Record{...})
    return nil
}
```

Perhatikan tanda <= : Token yang sama persis (duplikat) wajib ditolak

Setiap mutasi yang sah harus memiliki nomor token yang lebih tinggi secara absolut

Pemeriksaan token dan append record berada di dalam satu blok s.mu.Lock()

Menghilangkan window of vulnerability akibat race condition antar worker

Reject writes if incomingToken <= lastSeenToken; storage is the final monotonic gatekeeper.

Loop Pemilihan Kandidat & Simulasi Pause

Bagaimana kandidat bereaksi terhadap interval waktu dan penundaan eksekusi

```
for {
    select {
    case <-ctx.Done(): return
    case now := <-ticker.C:
        n.mu.Lock()
        if now.Before(n.pauseUntil) { n.mu.Unlock(); continue }
        if n.state == StateLeader {
            renewed, err := n.coord.Renew(...)
            if err != nil { n.state = StateFollower; n.currentLease = nil }
        } else {
            acquired, err := n.coord.Acquire(...)
            if err == nil { n.state = StateLeader; n.currentLease = acquired }
        }
        n.mu.Unlock()
    }
}
```

n.pauseUntil = time.Now().Add(duration) menahan loop tanpa intervensi OS sinyal

Mensimulasikan full stop-the-world GC pause atau delay partisi kabel jaringan

Periodic election loops react to lease deadlines while pause markers freeze execution safely.

Kronologi Kegagalan & Penolakan Stale Write

Rekonstruksi detik demi detik pencegahan korupsi data di shared resource

URUTAN WAKTU EKSEKUSI

t0: Node A acquire lease -> Token 1 diterbitkan -> Node A jadi Leader
t1: Node A menulis 'record-1' dengan Token 1 -> Storage menerima (lastSeen=1)
t2: Node A mengalami Pause 350ms (TTL=200ms) -> Heartbeat berhenti terkirim
t3: TTL habis di t0+200ms -> Koordinator menandai lease Node A kedaluwarsa
t4: Node B acquire lease -> Token 2 diterbitkan -> Node B jadi Leader baru
t5: Node B menulis 'record-2' dengan Token 2 -> Storage menerima (lastSeen=2)
t6: Node A terbangun dari pause di t2+350ms -> Mengira masih memegang token 1
t7: Node A menulis dengan Token 1 -> Storage REJECT: Token 1 <= lastSeen 2!

Pada t6, Node A dan Node B sempat sama-sama meyakini dirinya adalah LEADER
Sistem terdistribusi asinkron tidak dapat mencegah persepsi ganda sementara ini
Namun integritas data tetap 100% terjaga berkat rejection di FencedStorage

Transient dual-leader states are inevitable; fencing tokens render stale leaders powerless at the gate.

Eksekusi Demo: Akuisisi Kepemimpinan Awal

Node-A memenangkan kampanye awal dan melakukan penulisan berfencing sah

```
[DEMO] Memulai 2 kandidat bersaing untuk lease 'app-primary' (TTL: 200ms, Renew: 50ms)...
[STATUS] Node-A: LEADER | Node-B: FOLLOWER
[LEADER] Node-A memenangkan pemilihan, token: 1
[WRITE] Node-A menulis dengan token 1...
[SUCCESS] Storage menerima tulisan dari Node-A dengan token 1
[PAUSE] Mensimulasikan stop-the-world GC pause 350ms pada Node-A...
```

Node-A berhasil memperoleh lease pertama dan menerima FencingToken bernilai 1

Storage mengizinkan penulisan record pertama karena token 1 > lastSeenToken 0

Nilai lastSeenToken pada FencedStorage diperbarui menjadi 1

Node-B tetap berstatus FOLLOWER karena Acquire ditolak dengan ErrLeaseHeld

The winning candidate obtains token 1 and successfully writes to storage initialized at 0.

Eksekusi Demo: Failover & Penolakan Mutasi Stale

Node-B mengambil alih dan request zombie Node-A digagalkan oleh storage

```
[FAILOVER] Node-B mengambil alih kepemimpinan, token baru: 2
[STATUS] Node-A: LEADER (pause) | Node-B: LEADER
[WRITE] Node-B menulis dengan token 2...
[SUCCESS] Storage menerima tulisan dari Node-B dengan token 2
[RESUME] Node-A terbangun dari pause dan mencoba menulis dengan token lama (1)...
[FENCING DEFENSE] Storage MENOLAK tulisan dari Node-A!
  Error: stale fencing token: token 1 <= last seen 2 (author: Node-A)
[VERIFIKASI] Storage records berisi 2 entri yang sah, split-brain berhasil dicegah!
```

Node-B sukses memvalidasi token 2 di koordinator setelah lease Node-A expired

Tulisan Node-A pasca-bangun ditolak secara presisi oleh ErrStaleFencingToken

Tidak ada data corrupt atau overwriting yang terjadi di dalam penyimpanan bersama

Failover elevates candidate B to token 2, causing the awakened leader A's write to be denied.

Bukti Pengujian Otomatis: 4/4 Test Suites Lulus

Verifikasi formal akuisisi sewa, penolakan token, failover, dan balapan paralel

```
=== RUN   TestCoordinatorLeaseAcquisitionAndRenewal
--- PASS: TestCoordinatorLeaseAcquisitionAndRenewal (0.25s)
=== RUN   TestFencedStorageRejectsStaleTokens
--- PASS: TestFencedStorageRejectsStaleTokens (0.00s)
=== RUN   TestLeaderElectionFailoverAndSplitBrainDefense
--- PASS: TestLeaderElectionFailoverAndSplitBrainDefense (0.28s)
=== RUN   TestConcurrentElectionRace
--- PASS: TestConcurrentElectionRace (0.10s)
PASS: 4 passed in 5 packages (total time: 2.000s, zero race warnings)
```

TestCoordinatorLeaseAcquisitionAndRenewal: Memvalidasi renewal dan kenaikan revision

TestFencedStorageRejectsStaleTokens: Menolak pengulangan token dan token mundur

TestLeaderElectionFailoverAndSplitBrainDefense: Skenario full lifecycle pause & failover

TestConcurrentElectionRace: 5 kandidat bersaing simultan, tepat 1 leader terpilih

Pengujian dijalankan dengan flag go test -race ./... tanpa terdeteksi data race apa pun

Unit, failover, and race tests verify the single-leader invariant and monotonic fencing.

Debat Redlock: Martin Kleppmann vs antirez

Mengapa distributed lock tanpa fencing token tidak memadai untuk korektivitas

KRITIK MARTIN KLEPPMANN (2016)

Lock berbasis waktu di Redis (Redlock) mengasumsikan model waktu sinkron semu
GC pause, swap delay, atau lompatan jam (NTP shift) dapat membuat lock kadaluwarsa
Klien yang tidak sadar lock-nya habis akan menulis ke storage dan merusak data
Kesimpulan: Lock service saja tidak cukup; storage wajib melakukan fencing token

TANGGAPAN SALVATORE SANFILIPPO (ANTIREZ)

Redlock dirancang praktis untuk menjamin mutual exclusion probabilistik tinggi
Jika resource backend mendukung fencing token, algoritma koordinasi apa pun jadi aman
Redlock tetap sangat efisien untuk tugas yang tidak membutuhkan strict correctness

Gunakan lock sederhana (Redis) untuk optimasi efisiensi (pencegahan duplikasi job)

Gunakan konsensus terbukti (etcd/ZooKeeper) + fencing token untuk data kritis

Distributed locks only optimize efficiency; safety requires monotonic fencing at the resource layer.

Perbandingan Implementasi Sistem Terdistribusi

Bagaimana etcd, ZooKeeper, Consul, dan Redis mengelola pemilihan leader

Sistem	Mekanisme Pemilihan	Fencing Token Asli	Sifat Konsensus
etcd	Lease + Campaign Keys	Revision (int64)	Raft (Strict)
ZooKeeper	Ephemeral Sequentials	zxid / Node Version	ZAB (Strict)
Consul	Session TTL + Locks	ModifyIndex (uint64)	Raft (Strict)
Redis	SET NX EX (Redlock)	Tidak ada bawaan	Asynchronous

Revision pada etcd adalah counter 64-bit global yang monotonik bertambah di setiap mutasi

Kandidat dapat menggunakan revision key saat lease diberikan sebagai fencing token

ZooKeeper transaction ID (zxid) mencerminkan tatanan kausal total dari seluruh proposal

etcd provides revisions, ZooKeeper provides zxids; both offer native monotonic fencing tokens.

Pentingnya Jam Monotonik: Hindari NTP Steps

Mengapa wall-clock dilarang keras untuk pengukuran durasi lease

BAHAYA WALL-CLOCK (REAL TIME)

Wall-clock merefleksikan waktu kalender manusia (UTC / local time)
Sinkronisasi Network Time Protocol (NTP) dapat menyebabkan waktu melompat maju atau mundur
Lompatan mundur membuat lease tampak aktif selamanya; lompatan maju mematikan lease prematur

MONOTONIC CLOCK DI RUNTIME GO

Go runtime secara otomatis menyertakan monotonic reading pada panggilan time.Now()
Operasi durasi seperti time.Since(t) atau now.Before(expiry) membaca os monotonic timer
Tidak terpengaruh oleh penyesuaian waktu sistem operasi maupun leap second

Gunakan Monotonic Clock untuk internal expiry, renew timeouts, dan heartbeat loop

Gunakan Wall-Clock HANYA untuk pencatatan log audit manusia dan timestamp tampilan

Always calculate lease deadlines with monotonic timers; wall-clock shifts corrupt time checks.

Mencegah Herd Effect Saat Failover

Strategi antrian teratur agar ratusan follower tidak membanjiri koordinator

PROBLEMATIKA THUNDERING HERD

Jika 100 node serentak melakukan polling Acquire saat leader mati, koordinator overload
Lonjakan network traffic dan lock contention menurunkan responsivitas kluster

STRATEGI 1: DIRECT PREDECESSOR WATCH

Pola klasik ZooKeeper: Setiap kandidat membuat ephemeral sequential node
Kandidat nomor N HANYA memantau (watch) event penghapusan pada node nomor N-1
Saat leader gugur, hanya node berikutnya dalam antrean yang terbangun dan berpromosi

Jika menggunakan polling, tambahkan jitter acak pada durasi tunggu kandidat

Mendistribusikan waktu kedatangan request akuisisi secara merata di timeline

Watch immediate predecessors or add randomized jitter to eliminate thundering herd storms.

Observabilitas & Metrik Penolakan Fencing

Menjadikan ErrStaleFencingToken sebagai indikator kesehatan sistem

METRIK UTAMA YANG HARUS DIUKUR

fencing_token_rejections_total: Counter tulisan yang ditolak oleh resource gate
leader_lease_renew_duration_seconds: Latensi round-trip pembaruan heartbeat
leader_active_duration_seconds: Berapa lama seorang leader mampu mempertahankan jabatan
candidate_state: Gauge status node (0=Follower, 1=Candidate, 2=Leader)

SIGNIFIKANSI REJECTION ALERT

Satu kali penolakan stale write adalah bukti keberhasilan mekanisme pertahanan split-brain
Namun lonjakan penolakan menandakan adanya degradasi performa: GC pause parah atau jaringan flap
Gunakan rejection rate sebagai Early Warning System sebelum kegagalan hardware meluas

Catat setiap kejadian penolakan dengan menyertakan author ID, token lama, dan token aktif

A fencing rejection proves safety worked, but frequent rejections signal severe latency spikes.

Checklist Kesiapan Produksi Leader Election

9 poin verifikasi arsitektur sebelum menggelar sistem koordinasi ke live cluster

CHECKLIST PENERAPAN KOORDINASI

[ ] TTL lease dikalibrasi berdasarkan p99 durasi GC pause dan network RTT
[ ] Margin interval renewal diatur minimal 3× lebih cepat dari durasi TTL
[ ] Koordinator menerbitkan token monotonik strictly increasing di setiap lease baru
[ ] Resource penyimpanan menerapkan check-and-set atomik (token <= lastSeen -> REJECT)
[ ] Semua logika durasi dan timeout menggunakan Monotonic Clock
[ ] Leader melepaskan sewa secara sukarela (Release) saat proses shutdown anggun
[ ] Metrik dan alerting aktif untuk pemantauan fencing_token_rejections
[ ] Penanganan herd effect (watch predecessor / jitter backoff) telah diuji
[ ] Lulus pengujian konkurensi race detector (go test -race)

Jangan pernah mempercayai klien untuk menentukan apakah dirinya masih sah sebagai leader

Pastikan shared resource memiliki otoritas mutlak menolak tulisan basi

Calibrate TTL margins, enforce monotonic fencing at the resource, and verify under race tests.

Batasan & Lingkup Implementasi Lab 30

Membedakan simplifikasi edukasi dengan kebutuhan enterprise sesungguhnya

IN-MEMORY LINEARIZABILITY

Koordinator lab menggunakan map in-memory yang dilindungi Go sync.Mutex
Membuktikan kebenaran algoritma fencing tanpa perlu menjalankan cluster etcd multi-node
Tidak menyimulasikan crash koordinator atau kehilangan state akibat proses restart

SIMULASI PAUSE LOKAL

Penundaan eksekusi dilakukan via field pauseUntil bukan stop-the-world signal OS nyata
Tidak memodelkan Byzantine faults atau manipulasi jam sistem yang disengaja
Parameter lab (TTL 200ms, Renew 50ms) dioptimalkan untuk kecepatan eksekusi demo

Invarian single-leader terbukti deterministik pada pengujian konkuren 5 node

Pertahanan split-brain melalui penolakan token terbukti 100% menangkal korupsi data

This lab proves algorithmic correctness in-memory; production relies on consensus engines like etcd.

Tiga Prinsip Inti Koordinasi Terdistribusi

Pedoman fundamental bagi software engineer dalam mendesain arsitektur cluster

PRINSIP 1: WAKTU ADALAH ILUSI DALAM DISTRIBUTED SYSTEMS

Jangan pernah mengandalkan jam fisik atau durasi absolut untuk keamanan data mutlak
Lease temporal hanya memberikan mutual exclusion probabilistik
Ubah batas waktu menjadi tatanan logis monotonik yang dapat diverifikasi

PRINSIP 2: VALIDASI HARUS BERADA DI RESOURCE LAYER

Klien yang mengalami pause tidak pernah tahu bahwa dirinya sudah kedaluwarsa
Klien zombie akan selalu merasa dirinya adalah leader yang sah
Hanya lapisan penyimpanan bersama yang dapat menjadi gerbang validasi akhir

Terapkan aturan perbandingan tegas: incomingToken > lastSeenToken

Tolak token yang sama (duplikat) maupun token masa lalu yang lebih rendah

Kunci atomisitas check-and-set menjamin integritas data dari race condition

Time is unreliable; safety demands monotonic ordering validated atomically at the shared resource.

Ringkasan Lab 30: Leader Election with Fencing Tokens

Leases grant temporary authority; fencing tokens enforce immutable safety at the storage gate.

HASIL VERIFIKASI UTAMA

Single-leader invariant terbukti deterministik di bawah balapan 5 kandidat simultan

Failover otomatis sukses mengambil alih kepemimpinan saat heartbeat leader terhenti

Shared storage menolak mutasi stale dari leader lama dengan ErrStaleFencingToken

4 unit dan race detector test suites lulus sempurna (go test -v -race ./...)

REPOSITORY

https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/30-leader-election

Lukman / lukman-ss

MENTAL MODEL UTAMA

A lease grants time-bounded authority; a fencing token converts that temporal authority

into a monotonic ordering that shared storage enforces — rejecting stale zombie writes.

RESOURCES & AUDIT

Repository: https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/30-leader-election

Commit SHA: e963675c67e7d518e19a07ad930dbdb5be73fff1

Jalankan Demo: go run ./cmd/demo (di folder lab 30)

Jalankan Test: go test -v -race ./... (di folder lab 30)